

Weather Forecast Dashboard Setup Guide

1. Required libraries

Backend (Python)

Use the Python requirements file to install backend dependencies.

- `fastapi`
- `uvicorn[standard]`
- `requests`
- `pandas`
- `joblib`
- `scikit-learn`
- `xgboost`
- `lightgbm`

You can install these from either:

- `requirements.txt` at the project root, or
- `backend/requirements.txt`

Frontend (Node.js)

The frontend dependencies are defined in `frontend/package.json`.

Key packages used by the UI:

- `react`
- `react-dom`
- `@vitejs/plugin-react`
- `vite`
- `typescript`
- `tailwindcss`
- `@tailwindcss/vite`
- `@mui/material`, `@mui/icons-material`
- `lucide-react`
- `recharts`
- `react-hook-form`
- `react-router`

- framer-motion
- react-dnd, react-dnd-html5-backend
- react-responsive-masonry
- react-day-picker
- clsx
- date-fns

The frontend is built with Vite and TypeScript.

2. How to install dependencies

Backend setup

From the project root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r backend/requirements.txt
```

If you prefer the root helper file, use:

```
pip install -r requirements.txt
```

Frontend setup

From the project root:

```
npm --prefix frontend install
```

Or manually:

```
cd frontend
npm install
```

3. How to run the software

Start backend

From the project root after activating the Python virtual environment:

```
python -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

This starts the FastAPI backend on `http://127.0.0.1:8000`.

Start frontend

From the project root:

```
npm --prefix frontend run dev
```

Or manually:

```
cd frontend
npm run dev
```

Then open the frontend URL shown by Vite, usually:

- `http://localhost:5173`

4. Backend endpoints (important routes)

The backend exposes these API endpoints:

- `GET /api/health` - health check
- `GET /api/provinces` - list of available provinces
- `POST /api/fetch-all` - synchronize weather datasets for all provinces
- `POST /api/exploration` - fetch exploratory snapshot data for a selected date
- `POST /api/predict` - predict rain/no-rain for a province

5. Frontend top-down view

Entry point

- `frontend/src/main.tsx`
 - Bootstraps the React app and mounts it into the browser.

Root application

- `frontend/src/app/App.tsx`
 - Root layout for the UI.
 - Controls the theme, error display, and page structure.
 - Connects `InputPanel`, `PredictionOutput`, and `ReporterMode`.

API client

- `frontend/src/app/weatherApi.ts`
 - Contains API calls used by the frontend.
 - `predictWeather()` submits prediction requests.
 - `fetchAllWeatherData()` calls `POST /api/fetch-all`.
 - `fetchExplorationSnapshot()` calls `POST /api/exploration`.

Configuration and shared types

- `frontend/src/app/weatherConfig.ts`
 - Defines supported provinces, algorithm options, horizon values.
 - Exports types and request payload definitions.

Main UI panels

- `frontend/src/app/components/InputPanel.tsx`
 - User input form for selecting province, horizon, algorithm, and mode.
 - Sends requests to backend when the user clicks **GENERATE PREDICTION** or **RUN EVALUATION**.
 - Also includes **SYNC 34 PROVINCES** to refresh dataset status.
- `frontend/src/app/components/PredictionOutput.tsx`
 - Shows the prediction result once the backend returns data.
 - Displays rain probability, accuracy, model details, and a daily result table.
- `frontend/src/app/components/ReporterMode.tsx`
 - Contains dataset information and embeds the exploratory analysis panel.
 - Provides a summary of the weather dataset and model results.
- `frontend/src/app/components/EDAPanel.tsx`
 - Exploratory Data Analysis panel.
 - Loads snapshot data for a selected date.
 - Renders distribution charts for temperature, humidity, cloud cover, wind, pressure, and rain.
 - Displays province-level daily weather rows and a selected province summary.

Supporting UI modules

- `frontend/src/app/components/TopNav.tsx`
 - Application top navigation bar.
 - Includes theme toggle and header branding.
- `frontend/src/app/components/FeedbackPanel.tsx`
 - Optional feedback section (currently present but not always enabled in App).

6. Example usage

Example: Run a prediction

1. Open the app in the browser.
2. Choose a province, horizon, and algorithm.
3. Click **GENERATE PREDICTION** for forecast mode.
4. The backend returns a prediction response and the frontend displays it in `PredictionOutput`.

Example: Run an evaluation

1. Switch mode to EVALUATE in InputPanel.
2. Select an evaluation date.
3. Click RUN EVALUATION.
4. The backend computes historical accuracy and shows a prediction table with `actual_rain` values.

Example: Explore historical data

1. In the dashboard, use the date picker inside EDAPanel.
2. Select a date between 2020-01-01 and yesterday.
3. The app loads aggregated province statistics and distribution charts.

7. Notes

- The backend loads data from `backend/load_weather_data/all` and uses trained models under `backend/ml_pipeline`.
- The frontend expects the backend API to be available at `http://127.0.0.1:8000`.
- If you need to change the backend URL, update `frontend/src/app/weatherApi.ts` or set `VITE_API_BASE_URL`.